Défis en Intelligence Artificielle. Reinforcement Learning (3)

Xavier Siebert

MathRO, Université de Mons

http://www.ig.fpms.ac.be/~siebertx





Menu

1. Comments on previous session
2. Approximate RL
3. Deep Reinforcement Learning
4. How to make Deep q-learning work

Possible implementation of q-learning

```
def __init__(self, **args):
  ReinforcementAgent.__init__(self, **args)
  self.qVals = util.Counter()

def getQValue(self, state, action):
  return self.qVals[(state, action)]

def getValue(self, state):
  actions=self.getLegalActions(state)
  if len(actions)==0:
      return 0.0
  values=[]
  for action in actions:
      values.append(self.getQValue(state, action))
  return max(values)

def update(self, state, action, nextState, reward):
  qSa=self.getQValue(state,action)
  sample=reward+self.discount*self.getValue(nextState)
  self.qVals[(state,action)]=(1-self.alpha)*qSa+self.alpha*sample
```

Possible implementation of q-learning

```
def getPolicy(self, state):
  actions=self.getLegalActions(state)
  allActions=[]
  if len(actions)==0:
      return None
  for action in actions:
      allActions.append((self.getQValue(state, action), action))
  bestActions=[pair for pair in allActions if pair == max(allActions)]
  bestActionPair=random.choice(bestActions)
  return bestActionPair[1]
```

Exploration / Exploitation

```
def getAction(self, state):
  """
    Compute the action to take in the current state.  With
    probability self.epsilon, we should take a random action and
    take the best policy action otherwise.  Note that if there are
    no legal actions, which is the case at the terminal state, you
    should choose None as the action.
  """
  legalActions = self.getLegalActions(state)
  action = None
  p=self.epsilon
  if util.flipCoin(p):
      action=random.choice(legalActions)
  else:
      action=self.getPolicy(state)
  return action
```

Menu

1 Comments on previous session

2 Approximate RL
- Motivations
- Value Function Approximation
- Features
- Learning

3 Deep Reinforcement Learning

4 How to make Deep q-learning work

Large-scale Renforcement Learning

Motivations for Approximate RL

- What happens if the table (s,a) is too large ?
  - Impossible to visit all states
  - Almost every state encountered will never have been seen before.
  - Impossible to keep everything in memory.
- Need a method that scales up and is able to generalize



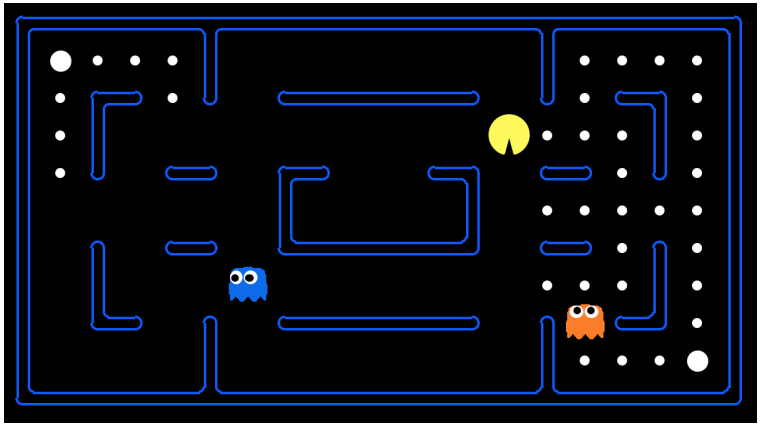
Large-scale Renforcement Learning

Motivations for Approximate RL

- Tic-Tac-Toe : 255 168 states
- Backgammon : $\approx 10^{20}$ states
- Chess : $\approx 10^{120}$ states
- Go : $\approx 10^{170} - 10^{360}$ states
- Robots : continuous state space

Large-scale Renforcement Learning

PacMan "MediumClassic"



- count the number of states $|\mathcal{S}|$:
  - PacMan's location (107 possibilities).
  - Location of each ghost (107^2 possibilities)
  - Pills remaining (2^2).
  - Locations still containing food : (2^{104} combinations) (Not all feasible because PacMan can't jump !)
- ... $|\mathcal{S}| \gg 107^3 = 1225043$

Approximate RL and generalization

PacMan example

- With "naïve" q-learning, those 3 states are different !





- need to generalize from examples ("knowledge transfer")
- $\rightarrow$ approximate RL $\approx$ function approximation

Value function approximation

- Approximating $v_\pi(s)$ from experience using a known policy π
- Parameterized functional form ($\boldsymbol{w} \in \mathbb{R}^d$) with $d << |\mathcal{S}|$

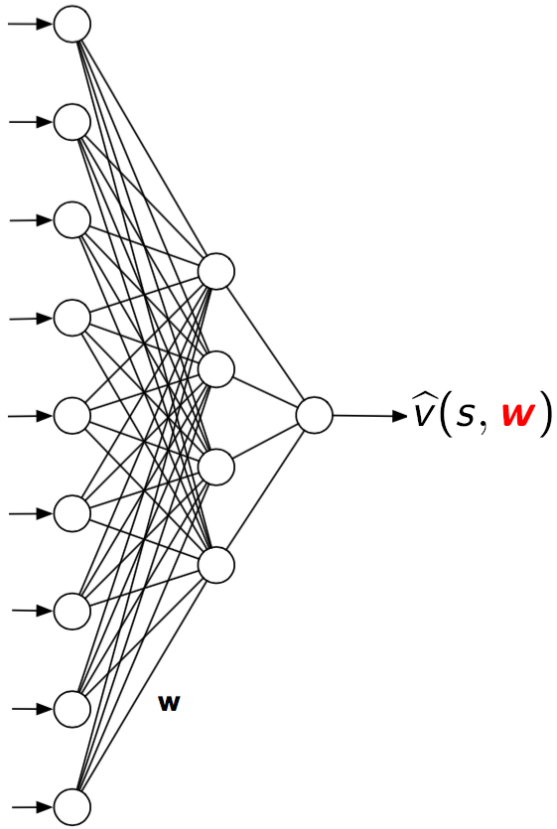
$$\boxed{v_\pi(s) \approx \widehat{v}(s, \boldsymbol{w})}$$

- Which function approximation ?
  - linear combination of features

$$\widehat{v}(s, \boldsymbol{w}) = \sum_{i=1}^{d} w_i f_i(s)$$

Value function approximation

- Which function approximation ?
  - (deep) neural network



q-Value function approximation

- Same principle for q-values

$$\widehat{q}(s, a, \boldsymbol{w}) = \sum_{i=1}^{d} w_i f_i(s, a)$$

- Advantage: experience is summarized with a few numbers w_i
- Drawback : many states have similar features but different values
  - example : $f_i(s, a)$ = number of ghosts nearby
    - ★ 2 ghosts close to PacMan : same side or not ?
    - ★ one ghost close to PacMan : eatable or not ?
- Looks like supervised learning, but data are
  - non-stationary : policy changes over time
  - non-iid : S_t and S_{t+1} correlated

Approximate q-learning

Features-based representations for PacMan

- Which features $f_i(s)$ to characterize the states ?
  - Distance d_f to closest ghost
  - Distance d_p to closest dot
  - Number n_f of ghosts
  - Is Pacman in a tunnel ? (0/1)
  - $1/d_p^2$
  - ...
- Which features $f_i(s, a)$ to characterize the q-states ?
  - action a gets PacMan closer to ghost
  - action a gets PacMan closer to food
  - ...
- need "expert knowledge" about the problem

Remember : q-learning ("exact")

[Watkins, 1989]

- Current q-state $= q(s, a)$
- Sample (s, a, s', r)
- Compute the difference

$$\Delta(s, a) = [R(s, a, s') + \gamma \max_{a'} q(s', a')] - q(s, a)$$

- Update the q-value

$$q(s, a) \leftarrow q(s, a) + \alpha \Delta(s, a)$$

where α is the learning rate.

Approximate q-learning algorithm ("fitted")

[Gordon, 1996]

- Current q-state $= q(s, a)$
- Sample (s, a, s', r)
- Compute the difference

$$\Delta(s, a) = [R(s, a, s') + \gamma \max_{a'} q(s', a')] - q(s, a)$$

- Update the weights

$$w_i \leftarrow w_i + \alpha \Delta(s, a) f_i(s, a)$$

where α is the learning rate.

- Intuition : q-value $\searrow$: blame it on the features that were active !

Example : approximate q-learning for PacMan

- Let us take the following approximation :

$$q(s,a) = w_{\mathrm{DOT}} f_{\mathrm{DOT}}(s,a) + w_{\mathrm{GST}} f_{\mathrm{GST}}(s,a)$$

  where
  - $f_{\mathrm{DOT}} = 1/(\text{distance to closest dot})$
  - $f_{\mathrm{GST}} = 1/(\text{distance to closest ghost})$
- start with an estimation

$$w_{\mathrm{DOT}} = 4 \qquad w_{\mathrm{GST}} = -1$$

- the current state s is this:



Example : approximate q-learning for PacMan

- take action NORTH from the state s corresponds to the q-state (this is not an actual state, the ghosts have not moved)



  - closest dot at a distance of 2

$$f_{\mathrm{DOT}}(s, \mathrm{NORTH}) = \frac{1}{2}$$

  - closest ghost at a distance of 1

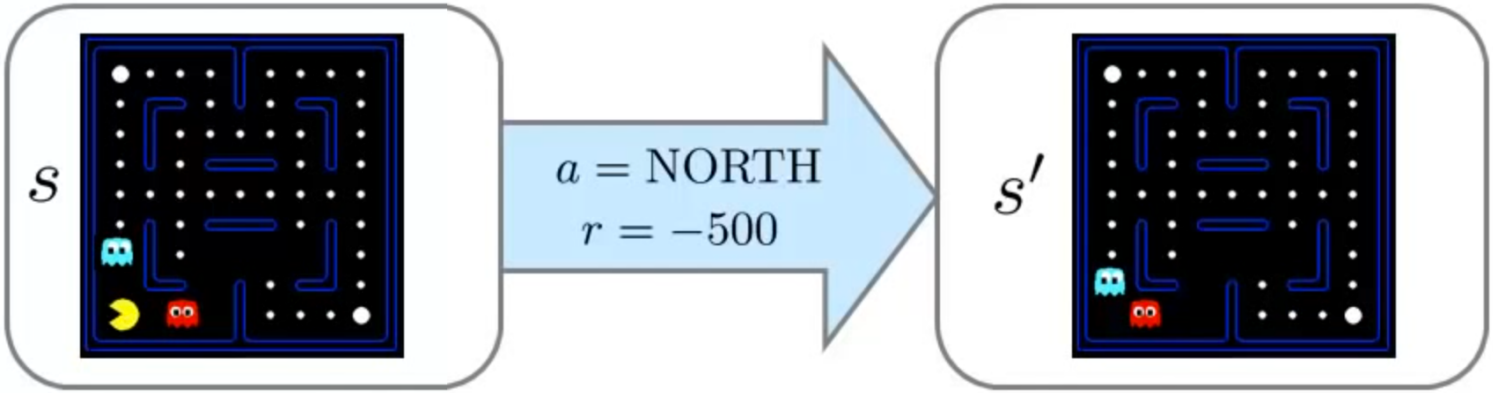
$$f_{\mathrm{GST}}(s, \mathrm{NORTH}) = 1$$

- with $w_{\mathrm{DOT}} = 4$ and $w_{\mathrm{GST}} = -1$, this leads to the current estimate

$$q(s, \mathrm{NORTH}) = 4 \times \frac{1}{2} - 1 \times 1 = 1$$

Example : approximate q-learning for PacMan

- take action NORTH from the state s results in the state s'



  - after transition, $q(s', a') = 0,\ \forall a'$ (PacMan died)
  - the reward for the transition is $R(s, a, s') = -500$
- we obtain the difference

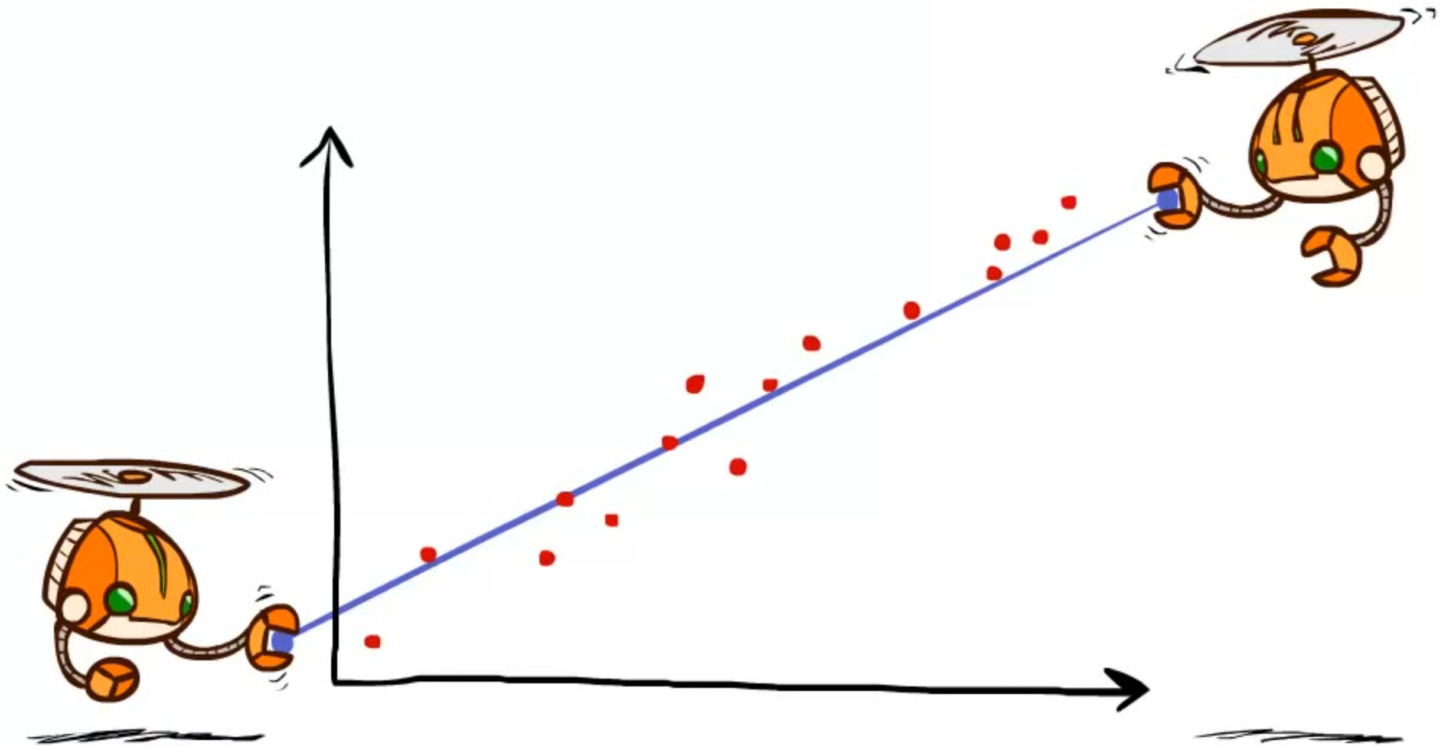
$$\Delta(s, a) = [R(s, a, s') + \gamma \underbrace{\max_{a'} q(s', a')}_{0}] - q(s, a) = -500 - 1 = -501$$

- we can thus compute the new values

$$w_{\text{DOT}} \leftarrow 4 + \alpha(-501)\frac{1}{2} \qquad w_{\text{GST}} \leftarrow -1 + \alpha(-501)1$$

Approximate q-learning

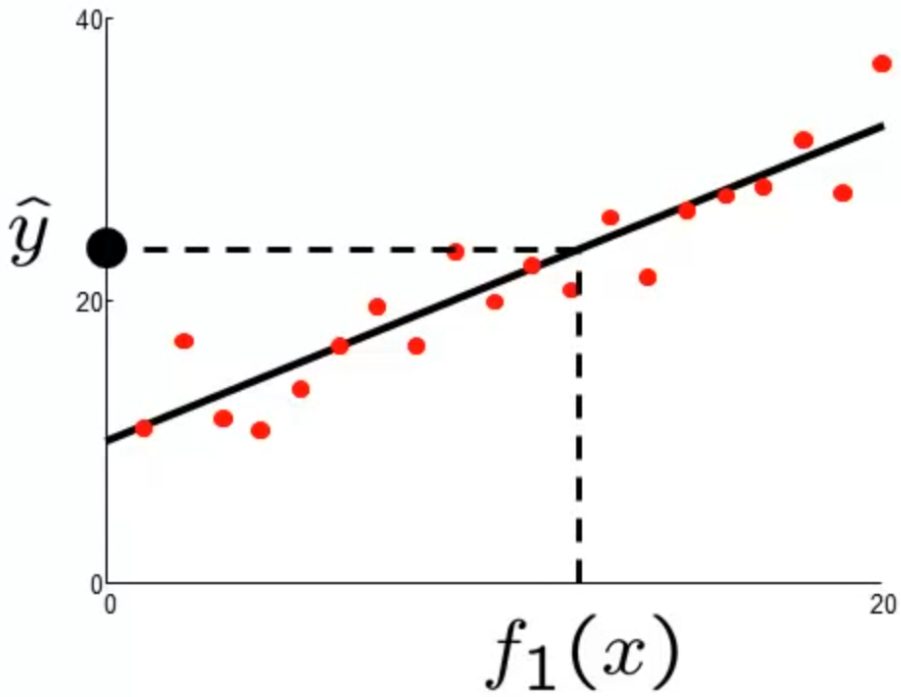
Link with linear regression



Approximate q-learning

Link with linear regression

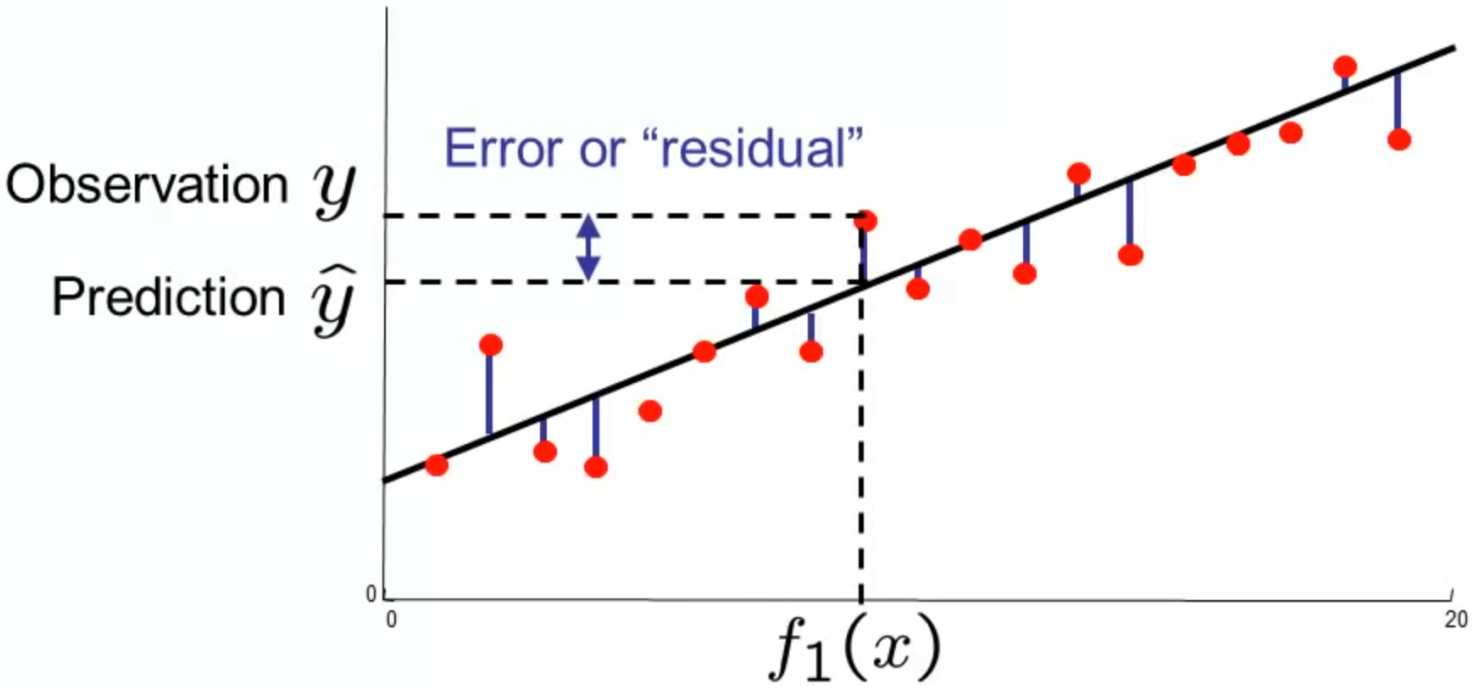
$$\widehat{y} = w_0 + w_1 f_1(x)$$



Approximate q-learning

Link with linear regression

$$\text{error} = \sum_i (y_i - \hat{y}_i)^2 = \sum_i (y_i - w_0 - w_1 f_1(x_i))^2$$



Approximate q-learning

Link with linear regression and gradient descent

- To get optimal w_1, compute the gradient

$$\frac{\partial \text{error}}{\partial w_1} = -2(y_i - w_0 - w_1 f_1(x_i))\ f_1(x)$$

- Correct the weights in the direction opposite to gradient

$$w_1 \leftarrow w_1 + \alpha(\underbrace{y_i}_{\text{target}} - \underbrace{(w_0 + w_1 f_1(x_i))}_{\text{prediction}}) f_1(x)$$

- Similar to q-update

$$w_i \leftarrow w_i + \alpha \Delta(s,a) f_i(s,a)$$

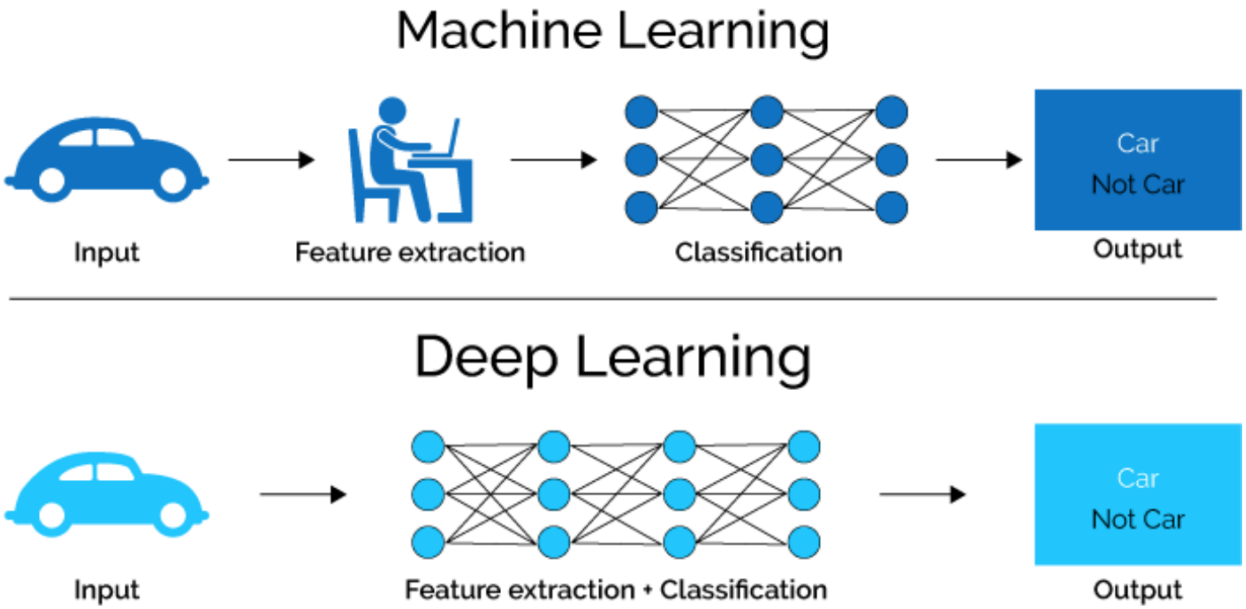
where $\Delta(s,a) = \underbrace{[R(s,a,s') + \gamma \max_{a'} q(s',a')]}_{\text{target}} - \underbrace{q(s,a)}_{\text{prediction}}$

Menu

1. Comments on previous session
2. Approximate RL
3. Deep Reinforcement Learning
   - Introduction
   - Not-So-Deep RL
   - Deep q-learning
4. How to make Deep q-learning work

Deep Reinforcement Learning

- Going further than function approximation
- No need to handcraft the features
- Uses Deep Neural Networks to learn the features



Source: towardsdatascience.com

Deep Reinforcement Learning

Example : Atari games

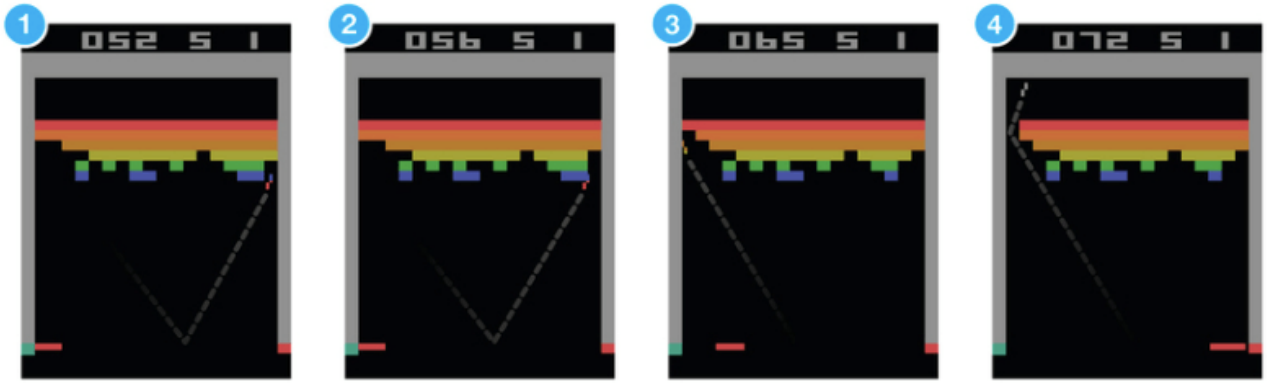
- *"Playing Atari with Deep Reinforcement Learning"* (NIPS 2013)
- First successes with 6 Atari games



- a deep RL agent can learn just by watching the pixels
- off-policy, model-free

Deep Reinforcement Learning

Example : Atari "Breakout" game

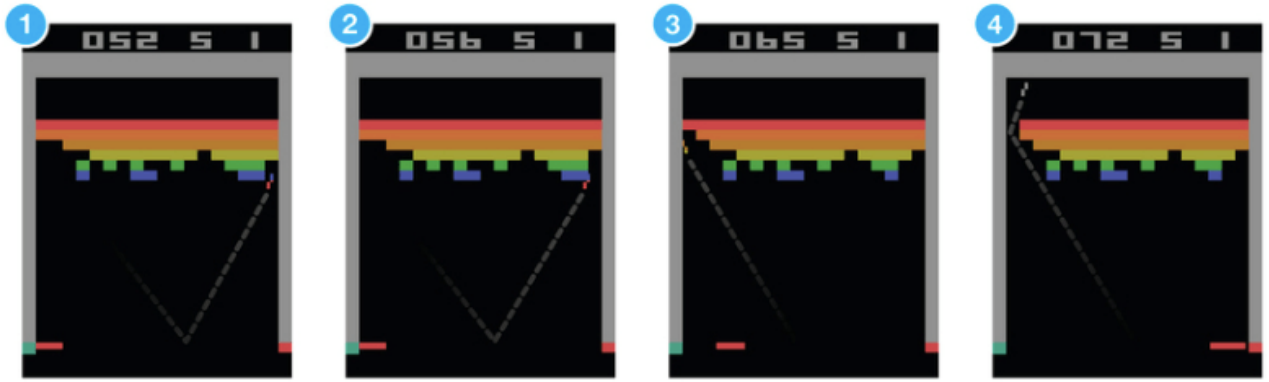


Atari Breakout game.
Image credit: DeepMind.

- Human-like learning : just watch the pixels !
- The same model plays all the games, beating human at 3 of them
- Demo: `https://www.youtube.com/watch?v=V1eYniJ0Rnk`
- *Human-level control through deep reinforcement learning* (Nature 2015)

Deep Reinforcement Learning

- Some numbers to illustrate the complexity of the task



Atari Breakout game.
Image credit: DeepMind.

- Suppose we use the last 4 images
  - 84×84 pixels
  - 256 grey levels
  - $\rightarrow 256 \times 84 \times 84 \times 4 \approx 1067970$ possible states
- many states never visited
- deep RL most useful in problems with high dimensional state-space.

Deep Reinforcement Learning

Reinforcement Learning for Atari

- Learning Atari took weeks of training

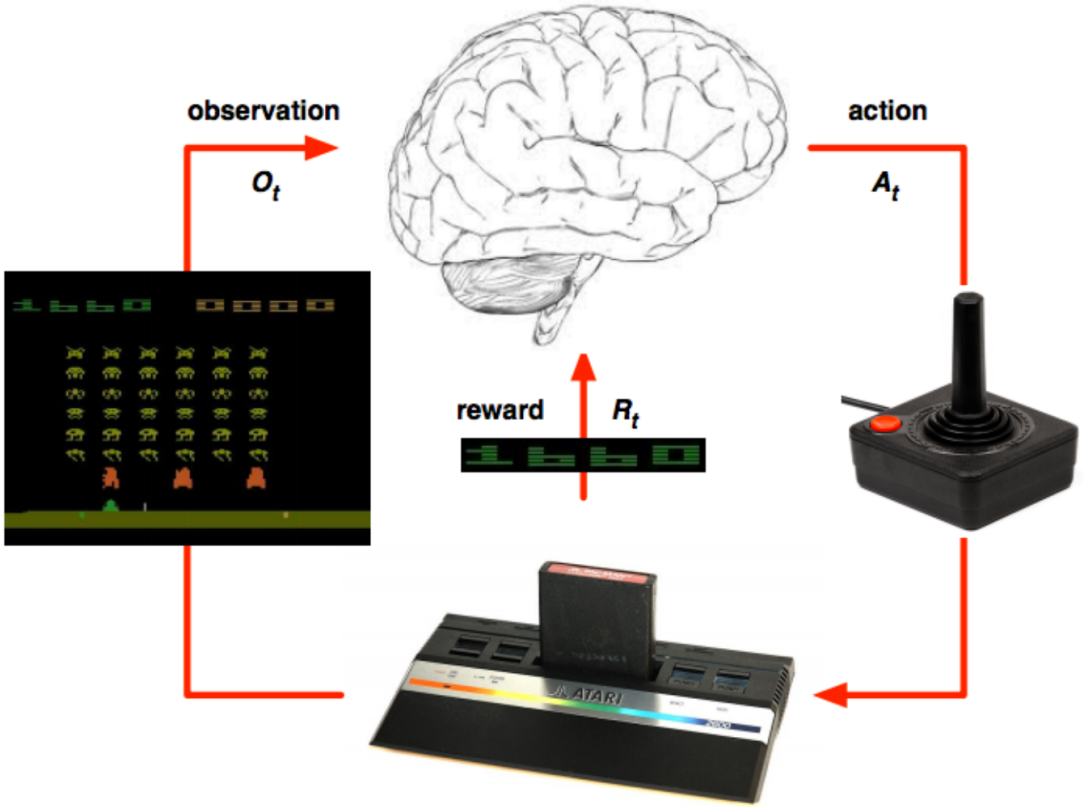


Image credit: Mnih *et al.*

Deep Reinforcement Learning

- Theoretical motivation: neural network = function approximator

Theorem: Universal Approximators [Hornik, 1989]

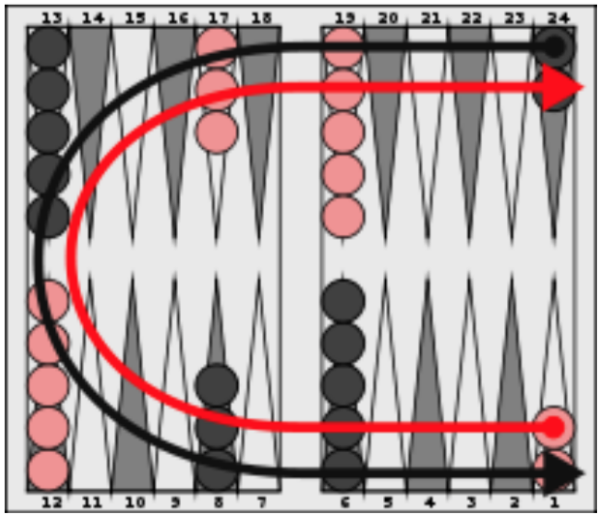
Any feedforward neural network with $n \geq 1$ hidden layers is able to approximate any measurable function in $\mathbb{R}^d$ (d finite) with an arbitrary precision $\epsilon > 0$.

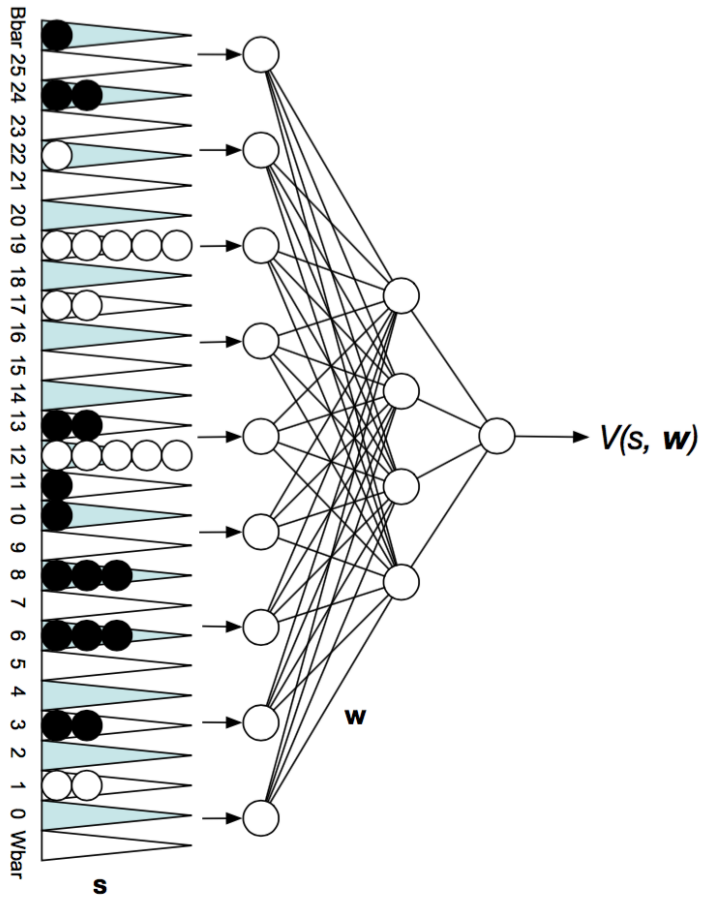
- Which function to approximate ?
  - policies $\pi(s)$
  - values $v(s)$
  - q-values $q(s, a)$
  - models

Not-So-Deep Reinforcement Learning

Example : TD Gammon (TD = temporal difference)

- Gerry Tesauro (1992, IBM Research Center) : one hidden layer





source: David Silver

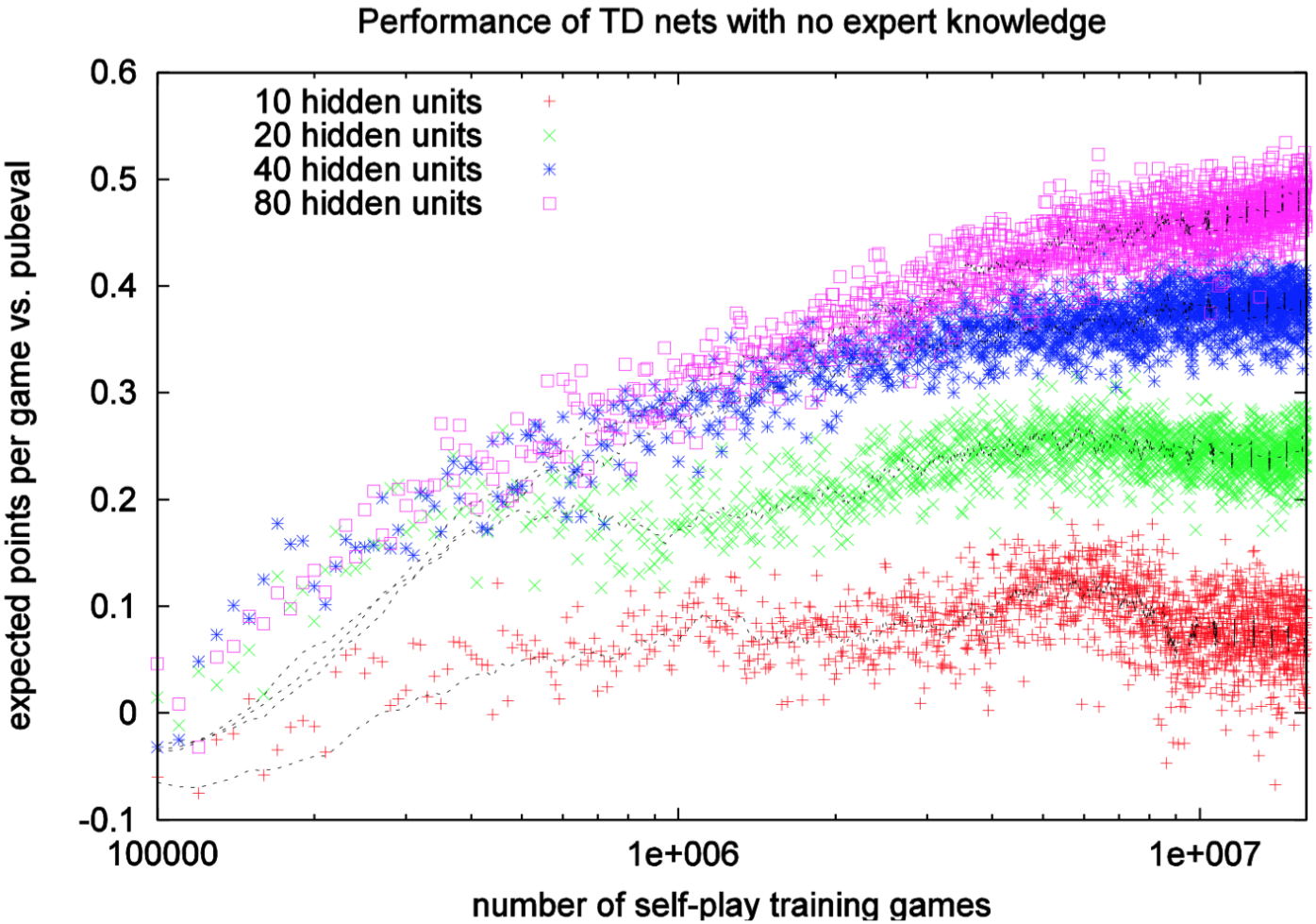
Not-So-Deep Reinforcement Learning

Example : TD Gammon

- Neural network builds its own features
- No expert knowledge
- Play against itself ($1,5 \times 10^6$ times), rewards are sparse :
  - $+100$ if win
  - -100 if loose
  - 0 otherwise
- Original TD Gammon : greedy policy improvement (no exploration)
- Defeated the world champion Luigi Villa 7-1

Not-So-Deep Reinforcement Learning

Example : TD Gammon (TD = temporal difference)



source: David Silver

Not-So-Deep Reinforcement Learning

Principle for q-value iteration

- Represent $q^\pi(s,a)$ with a deep network, whose weights are $\boldsymbol{w}$

$$q(s,a,\boldsymbol{w}) \approx q^\pi(s,a)$$

- To learn $q(s,a,\boldsymbol{w})$, define an objective function, e.g., least squares

$$\mathcal{L}(\boldsymbol{w}) = \mathbb{E}\left[\left(\underbrace{R(s,a,s') + \gamma \max_{a'} q(s',a',\boldsymbol{w})}_{\text{target}} - \underbrace{q(s,a,\boldsymbol{w})}_{\text{network}}\right)^2\right]$$

- Backpropagation with q-learning (stochastic) gradient descent

$$\nabla_{\boldsymbol{w}}\mathcal{L} = \mathbb{E}\left[\left(R(s,a,s') + \gamma \max_{a'} q(s',a',\boldsymbol{w}) - q(s,a,\boldsymbol{w})\right)\nabla_{\boldsymbol{w}} q(s,a,\boldsymbol{w})\right]$$

Not-So-Deep Reinforcement Learning

Approximation of $q(s, a)$

- Naïve approach



- Neural-fitted q-learning (NFQ) [Riedmiller 2005]



Not-So-Deep Reinforcement Learning

Approximation of $q(s, a)$

- Neural-fitted q-learning (NFQ) $\approx$ q-table approximation
- Parameterized representation of $q(s_t, a_t|\boldsymbol{w})$
- Select the best action by finding $\max_i q(s, a_i)$ in a single forward pass, because we have all the $q(s, a_i)$ as the output for a given s
- This setting only works for discrete q-space

Not-So-Deep Reinforcement Learning

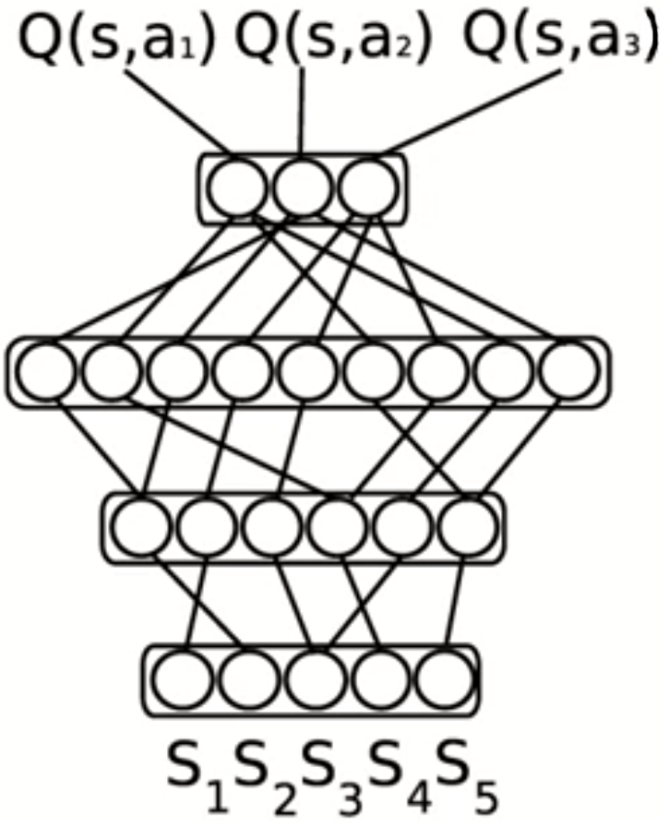
Potentiel problems

- updating the weights $\boldsymbol{w}$ also changes the target...
  - errors can propagate
  - convergence might be slow
  - can be unstable (no theoretical convergence guarantee)
- q-values can be overestimated because of the max operator
- (stochastic) gradient descent is great but not sample efficient
  - turn to batch learning
  - make more efficient use of the samples, instead of throwing them away
- care needs to be taken for proper learning

Deep Reinforcement Learning

Approximation of $q(s, a)$

- Deep q-learning Network = DQN [Mnih et al, 2015]



Deep Reinforcement Learning

Deep q-Network for Atari

- Learns to play from pixels only
- Same architecture for all games
  - input : pixels of the last 4 frames
  - output : $q(s, a_i)$ for $i = 1, \ldots, 18$ joystick/knobs positions
- Convolutional networks useful to represent local connectivity

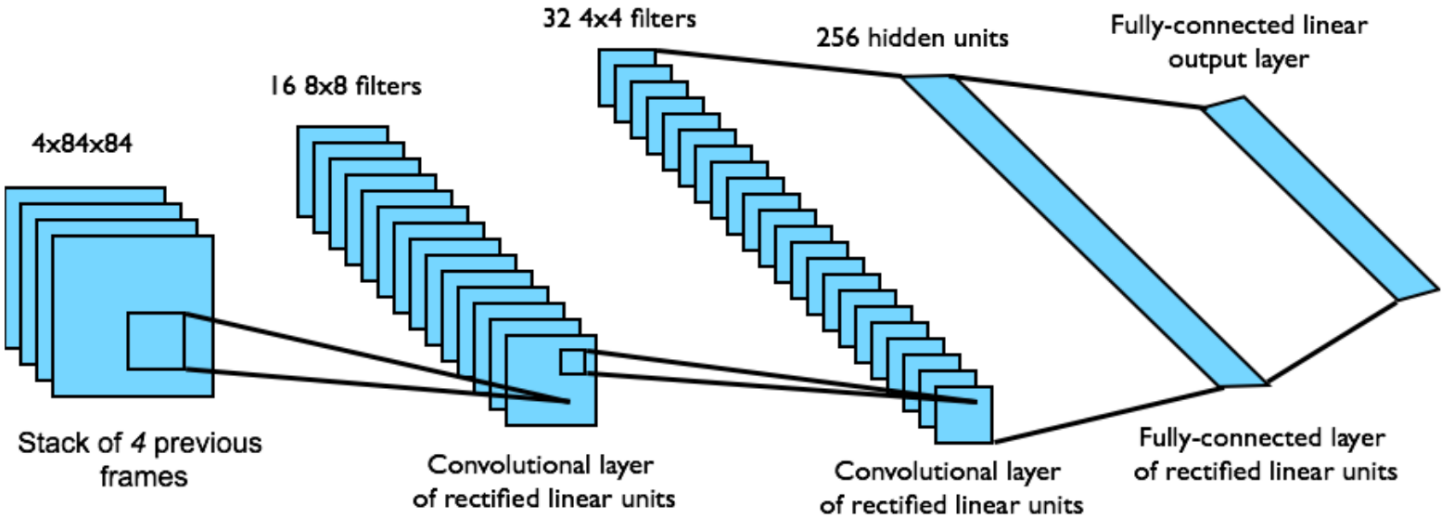


Image credit: Mnih *et al.*

Deep Reinforcement Learning

Deep Network for Atari

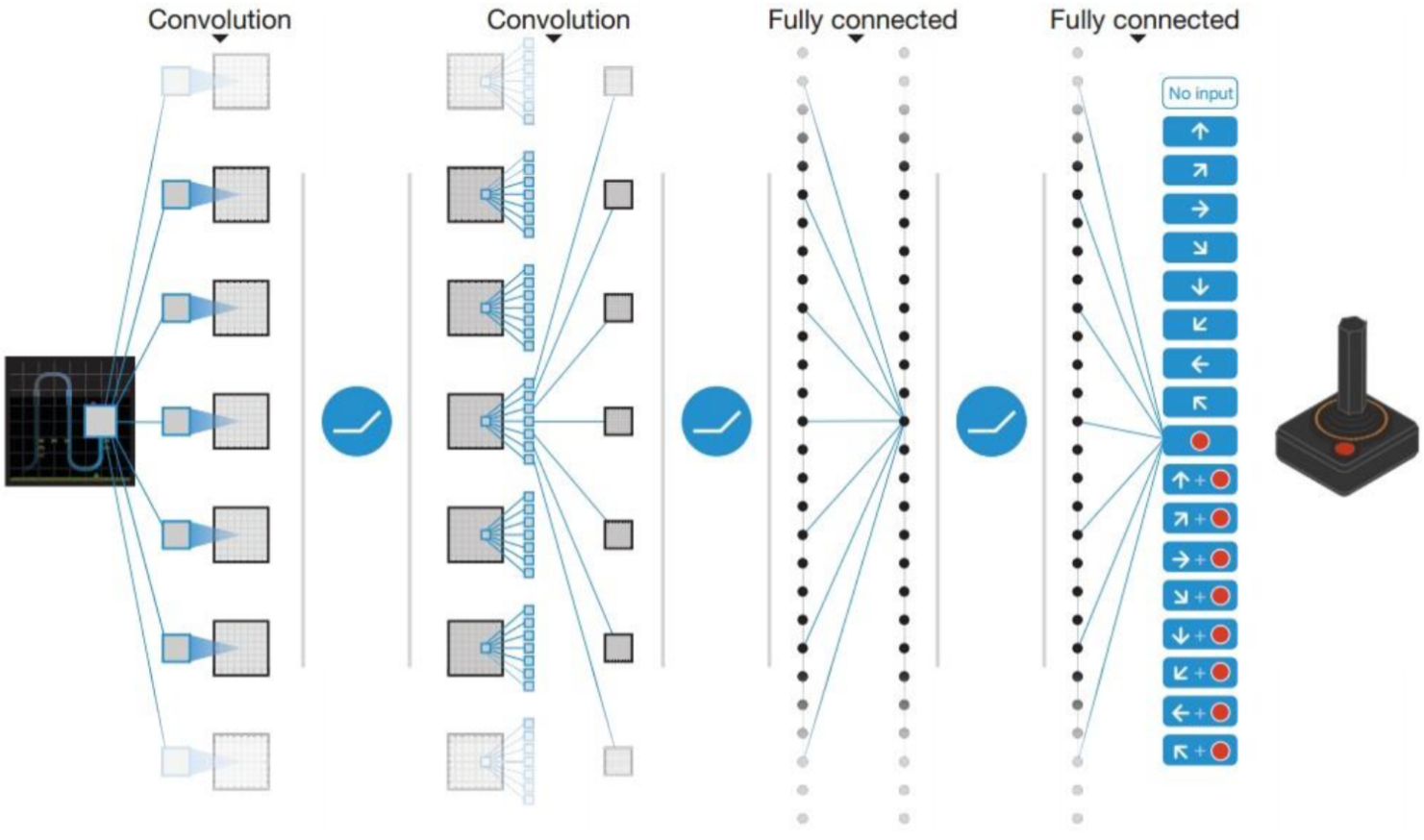


Image credit: Mnih *et al.*

- input layer = 84×84 pixels of last 4 frames
- output layer = 18 possible movements with joystick

Deep Reinforcement Learning

Deep Network for Atari

Layer	Input	Filter size	Stride	Num filters	Activation	Output
conv1	84x84x4	8x8	4	32	ReLU	20x20x32
conv2	20x20x32	4x4	2	64	ReLU	9x9x64
conv3	9x9x64	3x3	1	64	ReLU	7x7x64
fc4	7x7x64			512	ReLU	512
fc5	512			18	Linear	18

Image credit: Mnih *et al.*

- input layer $= 84 \times 84$ pixels of last 4 frames
- output layer $= 18$ possible movements with joystick

Deep Reinforcement Learning

Exemple : Atari

- Performances in various Atari games vs. human benchmark (!)

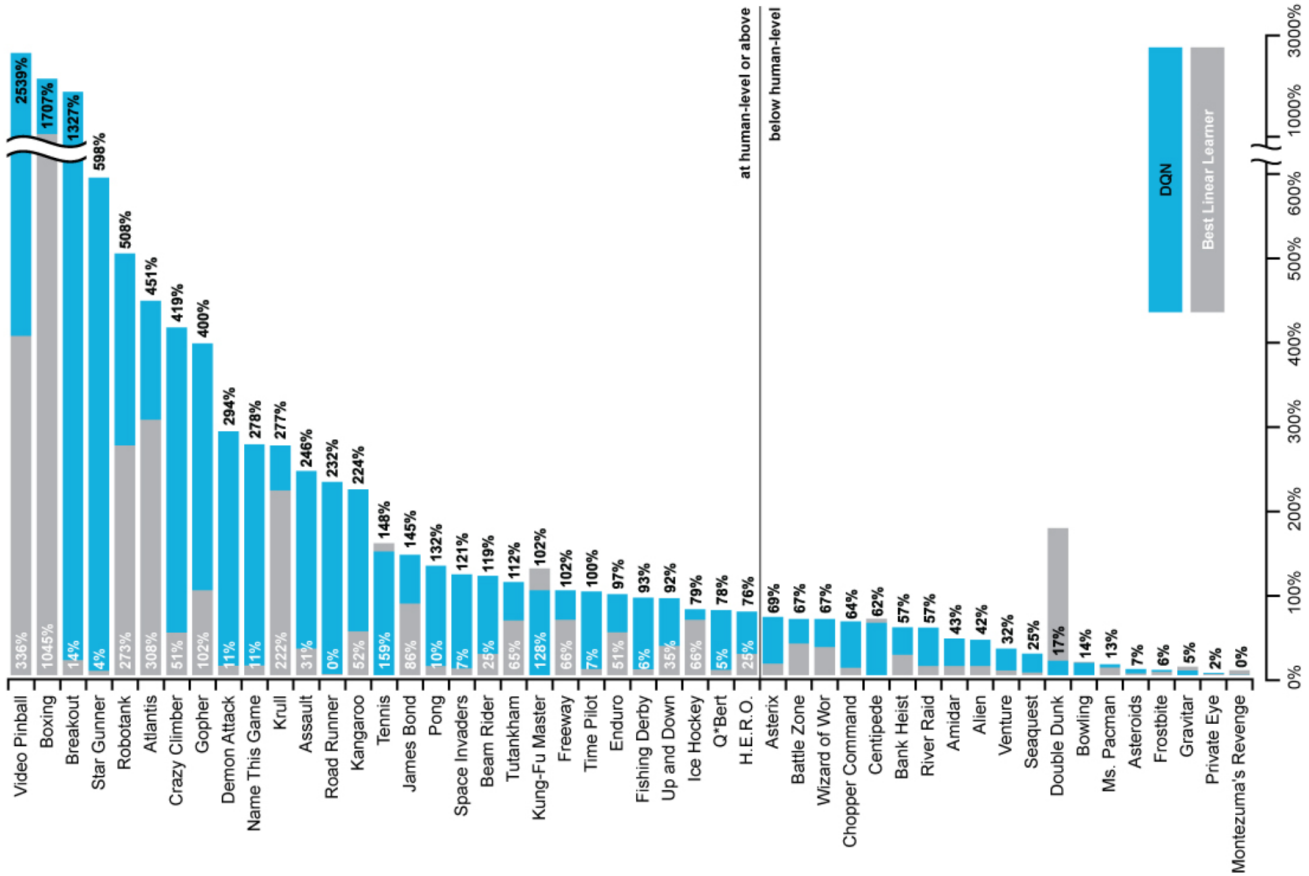


Image credit: Mnih *et al.*

Menu

1 Comments on previous session

2 Approximate RL

3 Deep Reinforcement Learning

4 How to make Deep q-learning work
- Tricks
- More Tricks
- Policy gradient methods
- Actor-critic architectures

Deep Reinforcement Learning

- Deep networks used in a particular way
  - supervised learning : known target
  - reinforcement learning : moving target
- Reinforcement learning creates its own, ever-shifting dataset
  - the network generates its own target values
  - the action choices of the network directly impact which states it will reach in its environment and therefore what it will have to learn about.
- Take a long time to train ($\approx$ 100 millions of frames for PacMan)
- Need to examine stability issues

Stability issues : potential problems

Problems : (Deep) q-learning can oscillate or diverge because

1. Data are sequential
   - Successive samples are **correlated** (non-iid).
   - The target depends on q

$$\underbrace{R(s,a,s') + \gamma \max_{a'} q(s',a',\boldsymbol{w})}_{\text{target}} - \underbrace{q(s,a,\boldsymbol{w})}_{\text{current}}$$

   so that learning is unstable (q could keep growing)
2. Policy changes rapidly with slight change in q-values.
   - Samples become useless each time we switch policy.
   - Policy may oscillate
3. The scale of rewards and q-values is unknown.
   - Gradients can be large, unstable when backpropagated

Stability issues : potential solutions

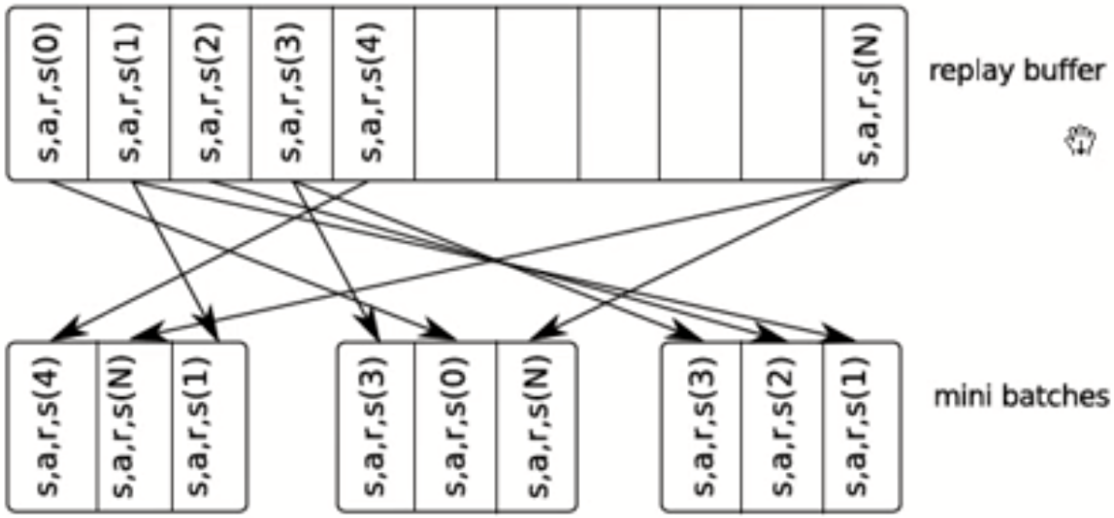
Some heuristics used with deep q learning

1. Experience replay : memorize a buffer of transitions (s_t, a_t, r_t, s_{t+1})
   - train the network on that memory of past policies (= "experience")
   - goal: break correlations in data
2. Freeze target q-network
   - Avoid oscillations
   - Break correlations between q-network and target network
3. Clip rewards or normalize network adaptively to sensible range
   - Robust gradients

(1) Experience Replay

To remove correlations in data, build dataset from agent's own experience

- Take action at according to ϵ-greedy policy
- Store N transitions $(s_t, a_t, r_{t+1}, s_{t+1})$ in replay memory $\mathcal{D}$
- Sample random mini-batches of transitions (s, a, r, s') from $\mathcal{D}$ shuffled
- These mini-batches cover a wide range of (less correlated) states



(1) Experience Replay

- Optimize MSE between q-network prediction and q-learning target

$$\mathcal{L}(\boldsymbol{w}) = \mathbb{E}_{s,a,r,s' \sim \mathcal{D}} \left[\left(\underbrace{R(s,a,s') + \gamma \max_{a'} q(s',a',\boldsymbol{w})}_{\text{target}} - \underbrace{q(s,a,\boldsymbol{w})}_{\text{prediction}} \right)^2 \right]$$

- Size of replay buffer has to be tuned
- Also allows for parallel computing of mini-batches

(1) Experience Replay

```
initialize replay memory D
initialize action-value function Q with random weights
observe initial state s
repeat
      select an action a
            with probability ε select a random action
            otherwise select a = argmax_a' Q(s,a')
      carry out action a
      observe reward r and new state s'
      store experience <s, a, r, s'> in replay memory D

      sample random transitions <ss, aa, rr, ss'> from replay memory D
      calculate target for each minibatch transition
            if ss' is terminal state then tt = rr
            otherwise tt = rr + γmax_a' Q(ss', aa')
      train the Q network using (tt - Q(ss, aa))^2 as loss

      s = s'
until terminated
```

(2) Fixed Target q-Network

- To avoid oscillations, fix parameters used in q-learning target
- Use 2 networks :
  - A : fixed during some time T
  - B : continually updated according to rewards
- Compute q-learning targets w.r.t. old, fixed parameters w^A

$$\mathcal{L}(\boldsymbol{w}^B) = \mathbb{E}_{s,a,r,s' \sim \mathcal{D}} \left[\left(\underbrace{R(s,a,s') + \gamma \max_{a'} q(s',a',\boldsymbol{w}^A)}_{\text{target network}} - \underbrace{q(s,a,\boldsymbol{w}^B)}_{\text{online network}} \right)^2 \right]$$

- Periodically (after period T) update fixed parameters $w^A \leftarrow w^B$
- Corresponds to "periods of supervized learning" (NFQ)

(3) Clip Reward/Value Range

- DQN clips the rewards to $[-1, +1]$
- prevents q-values from becoming too large
- ensures gradients are well-conditioned
- cannot tell the difference between small and large rewards...

Deep Reinforcement Learning

Example : Atari

- Effects of
  1. experience Replay
  2. fixed Target q-Network

	Q-learning	Q-learning + Target Q	Q-learning + Replay	Q-learning + Replay + Target Q
Breakout	3	10	241	**317**
Enduro	29	142	831	**1006**
River Raid	1453	2868	4103	**7447**
Seaquest	276	1003	823	**2894**
Space Invaders	302	373	826	**1089**

Source: David Silver

Deep Reinforcement Learning : more tricks

Double q-learning [Van Hasselt et al, 2015] :

$$\mathcal{L}(\boldsymbol{w}^B) = \mathbb{E}_{s,a,r,s' \sim \mathcal{D}} \left[\left(\underbrace{R(s,a,s') + \gamma \max_{a'} q(s', a', \boldsymbol{w}^A)}_{\text{target network}} - \underbrace{q(s,a,\boldsymbol{w}^B)}_{\text{online network}} \right)^2 \right]$$

- The target network (A) was used both
  - to calculate the q-values for each action
  - to select the $\max_{a'}$
- This can lead to overestimation of $q(s,a)$
  - ex: discrepancy between A and B cause them to recommend totally different actions for the same state

Deep Reinforcement Learning : more tricks

Double q-learning [Van Hasselt et al, 2015] :

- take away the target network's responsibility to determine the best action
  - target network simply generates the q values
  - let the online model decide which to use.

$$\underbrace{R(s,a,s') + \gamma \max_{a'} q(s', a', \mathbf{w}^A)}_{\text{target network}} - \underbrace{q(s,a,\mathbf{w}^B)}_{\text{online network}}$$

$$\Downarrow$$

$$\underbrace{R(s,a,s') + \gamma q(s', \arg\max_{a} q(s,a,\mathbf{w}^B), \mathbf{w}^A)}_{\text{target network}} - \underbrace{q(s,a,\mathbf{w}^B)}_{\text{online network}}$$

Deep Reinforcement Learning : more tricks

Dueling Architecture [Wang *et al.*, 2015]

- define an "advantage function" $A(s, a)$

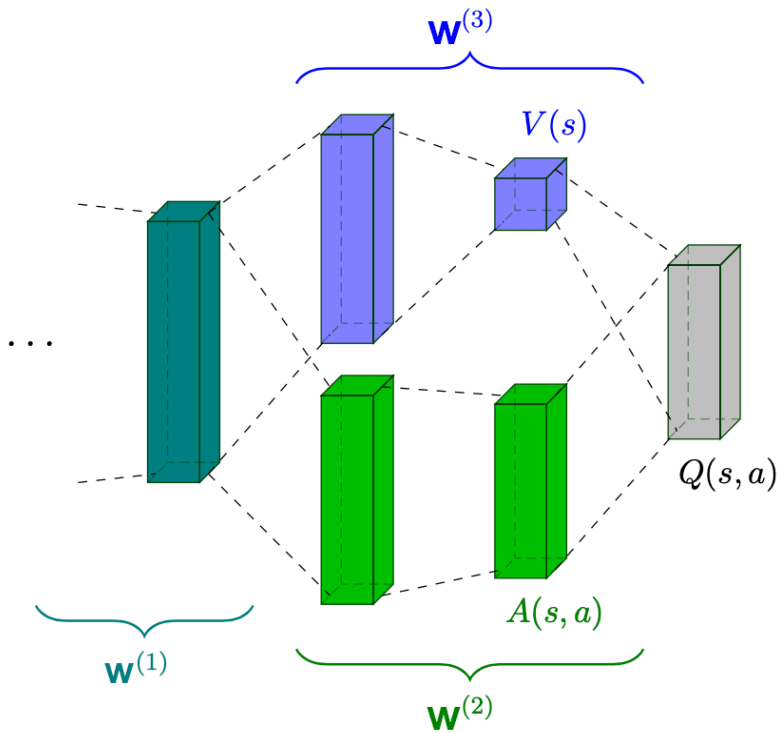
$$A^\pi(s, a) = q^\pi(s, a) - v^\pi(s)$$

- the q-value function can be split in two independent terms:
  - value function $v(s)$ = value of being in the current state,
  - advantage function $A(s, a)$ = relative advantage of each action
- advantage function $A(s, a)$ has lower magnitude than $q(s, a)$

Deep Reinforcement Learning : more tricks

Dueling Architecture [Wang *et al.*, 2015]

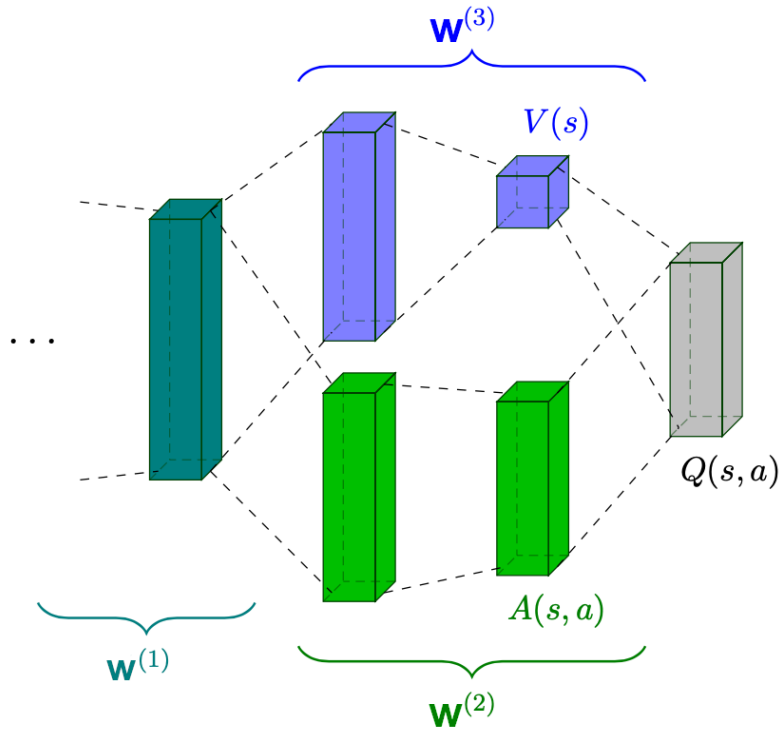
- $v(s)$ and $A(s,a)$ can be trained with different networks



- helps reducing the variance of the gradient estimator

Deep Reinforcement Learning : more tricks

Dueling Architecture [Wang *et al.*, 2015]

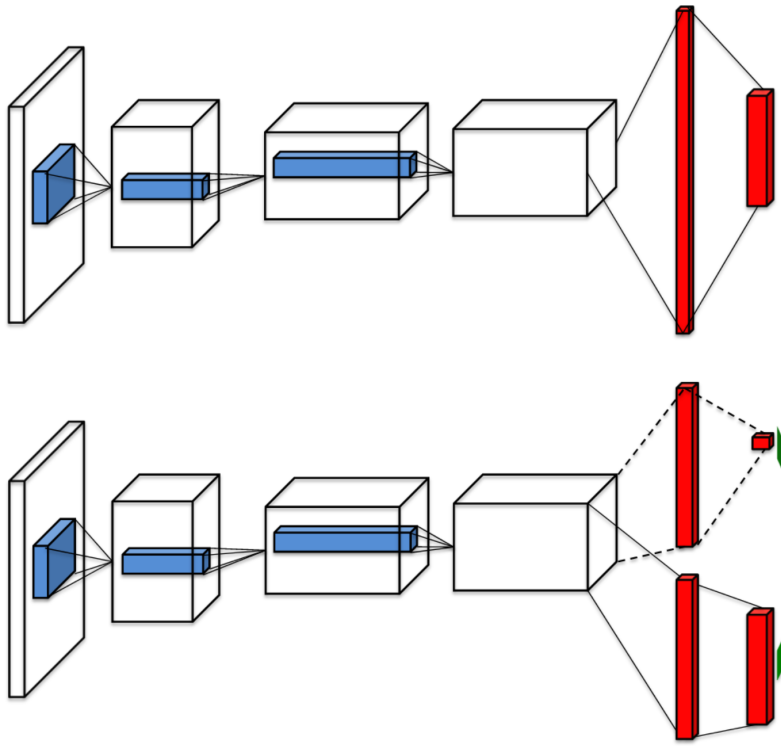


$$Q(s,a;\boldsymbol{w}^{(1)},\boldsymbol{w}^{(2)},\boldsymbol{w}^{(3)}) = V(s;\boldsymbol{w}^{(1)},\boldsymbol{w}^{(3)}) +$$

$$\left(A(s,a;\boldsymbol{w}^{(1)},\boldsymbol{w}^{(2)}) - \max_{a'} A(s,a';\boldsymbol{w}^{(1)},\boldsymbol{w}^{(2)}) \right)$$

Deep Reinforcement Learning : more tricks

Dueling Architecture



Source: Wang *at al.*, 2015

Deep Reinforcement Learning : more tricks

Prioritized experience replay [Schaul et al. 2016]

- So far the agent has been sampling from its replay buffer uniformly at random
- Would be better to learn from the experiences where it is the most wrong
- Use an additional data structure that keeps a record of the priority of each transition
- for example, priority = most inaccurate prediction ($\approx$ TD error)
- Experiences are then sampled proportional to their priorities

Deep Reinforcement Learning : more tricks

Noisy networks [Fortunato et al. 2017]

- how to balance exploration / exploitation ?
- noisy networks : add noise to the action selection
- inject noise into the weights of the neural network

Deep Reinforcement Learning

Policy gradient methods

- So far we approximated the (q-)value function using parameters θ,

$$\begin{aligned} v_\theta(s) &\approx v_\theta(s) \\ q_\theta(s,a) &\approx q_\theta(s,a) \end{aligned}$$

- A policy was generated directly from the value function
  - e.g. using ϵ-greedy
- this cannot explicitly learn stochastic policies
- alternative: directly parameterize the policy

$$\pi_\theta(s,a) = \mathbb{P}[a|s,\theta]$$

Deep Policies for continuous actions

- the policy can be build iteratively

$$\pi_{k+1}(s) = \arg\max_{q} q^{\pi_k}(s, a)$$

- Let a class Π of policies $\pi(s, \boldsymbol{u})$ parametrized by $\boldsymbol{u} \in \mathbb{R}^m$ (e.g., DNN)
- Define as objective function

$$q^{\pi}(s, \boldsymbol{u}) = \mathbb{E}[R_1 + \gamma R_2 + \gamma^2 R_3 + \ldots | \pi(s, \boldsymbol{u})]$$

- optimize $q^{\pi}(s, \boldsymbol{u})$ by stochastic gradient descent

$$\boldsymbol{u}^* = \arg\max_{\boldsymbol{u}} q^{\pi}(s, \boldsymbol{u})$$

- the policy gradient typically requires an estimate of a q-value function for the current policy

Actor-critic architectures

[Mnih et al., 2016]

- Hybrid between
  - Policy-based methods
  - Value-based methods
- in deep RL, we can use 2 networks
  - actor = Policy $\pi(s, \boldsymbol{u})$, determines the actions to take
  - critic = Value function $q(s, a, \boldsymbol{w})$, evaluates the actions
- the critic evaluates the actor using the difference

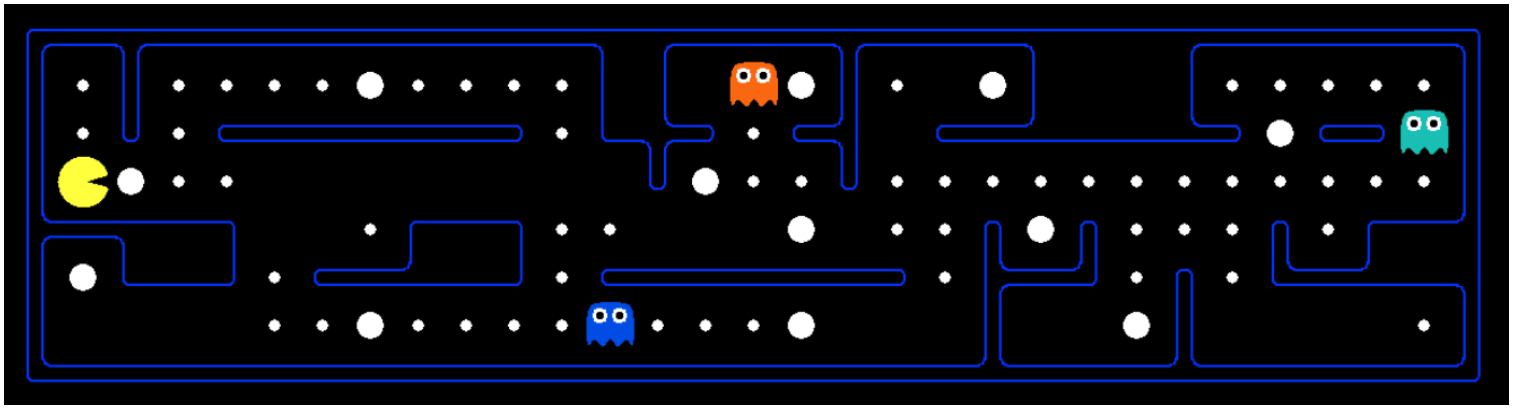
$$\delta_t = \underbrace{R_{t+1} + \gamma q(s_{t+1}, a)}_{\text{reward received}} - \underbrace{q(s_t, a)}_{\text{reward predicted}}$$

  - $\delta_t > 0$: action needs to be encouraged
  - $\delta_t < 0$: action needs to be discouraged
- gradient backpropagates from critic into actor

Actor-critic architectures

- very general framework
- can be used with any representation of the actor and the critic
  - deep learning : δ_t affects the weights of the network
- A2C = Advantage Actor-Critic
- A3C = Asynchronous Advantage Actor-Critic
  - perform the policy gradient on-policy without experience replay (no need to maintain a replay buffer)
  - multiple agents are executed in parallel and experience different parts of the environment
  - the actor-learners are trained asynchronously

Your turn . . .



- finish Section 2 (approximate q-learning)
- Section 3 (deep q-learning) : go as far as you can. . .

References

- Richard S Sutton and Andrew G Barto. Introduction to reinforcement learning. Vol. 135. MIT press Cambridge, 1998
- Leslie Pack Kaelbling, Michael L Littman, and Andrew W Moore. "Reinforcement learning: A survey". In: Journal of artificial intelligence research 4 (1996), pp. 237–285
- Volodymyr Mnih et al. "Playing atari with deep reinforcement learning". In: arXiv preprint arXiv:1312.5602 (2013)
- Volodymyr Mnih et al. "Human-level control through deep reinforcement learning". In: Nature 518.7540 (2015), p. 529
- David Silver et al. "Mastering the game of Go without human knowledge". In: Nature 550.7676 (2017), p. 354

Online references

- Courses
  - Berkeley CS 188
    `https://inst.eecs.berkeley.edu/~cs188/fa18`
  - Stanford CS 234
    `http://web.stanford.edu/class/cs234/index.html`
- Resources
  - `https://gym.openai.com/`
  - `https://selfdrivingcars.mit.edu/deeptraffic/`
  - T. Kulkarni, Deep Q Learning in Tensorflow for ATARI (2012)
    `https://github.com/mrkulk/deepQN_tensorflow`

One last thing

- Comments / Questions / Feedback are welcome, either live or at

xavier.siebert@umons.ac.be